

Get Started With

Gaming has become a major field in the computing world. It is the favourite entertainment for many users, but a challenging job for developers, because of the complexity involved. Pygame is a great set of Python modules that helps in building games with much less work. Since I'm learning about this module, I wanted to share my knowledge with you.

Earlier, developing a game was hard, because we needed to configure hardware according to the needs of the game. With the development of SDL (Simple DirectMedia Layer), developers don't need to mess with low-level programming any more. Pygame is the Python extension of SDL, with some additional benefits. Like SDL, it automatically covers the hardware interface, so you just need to code at a high level. It has replaced the older pySDL module. Pygame was released by Pete Shinnars, under the LGPL. One of its greatest strengths is the online community at <http://www.pygame.org/>. This link is worth a visit, as this community motivates programmers to build small games using this great API, through competitions and help.

Pygame should come pre-installed with Python. I am using the Ubuntu 10.04 32-bit desktop edition with Python 2.6.5. My Pygame version is 1.9.1. To check whether it is installed on your system, run the following in the interactive mode of your Python interpreter:

```
import pygame
pygame.ver
```

If installed, it will return the version of the Pygame module. If it shows an error, it has not installed. Download it from <http://pygame.org/download.shtml> and run the following steps as the root user to install it:

```
tar xzf pygame-1.9.1release.tar.gz
cd pygame-1.9.1release
python setup.py install
```

Hello World

Beginning with a 'Hello World' program is a tradition in the programming world. Let's follow the tradition, but with some extended/advanced code. Given below is a short script, *hello.py*, which will display 'Hello World' along with a background image. Find the file at http://www.linuxforu.com/article_source_code/may11/hello.py

```
import pygame #1
from pygame.locals import *
from sys import exit

pygame.init() #2
screen=pygame.display.set_mode((640, 480),0, 24) #3
pygame.display.set_caption("Hello World") #4
create=pygame.font.SysFont("comicsansms",30) #5
f=create.render("Hello World",True,(0, 0, 0),(255, 255, 255)) #6
img=pygame.image.load("simple.jpg").convert() #7
while True:

    for i in pygame.event.get(): #8
        if i.type==QUIT:
            exit()

    screen.blit(img,(0, 0)) #9
    screen.blit(f,(200, 200))
    pygame.display.update() #10
```

The output of this program can be seen in Figure 1.

Let's try to understand the output by following the code, step by step.

1. We *import pygame* so that we can use the module's functionality.
2. Pygame is initialised. Manual initialisation of resources leads to faster processing.
3. Everything in Pygame runs on surfaces; so we create a display surface and set its properties using the *set_mode* function, where the first argument is a tuple consisting of height and width as elements, the second a flag which is by default zero, and the third the colour information (8, 16, 24, 32-bit).
4. We set the caption of the created display to 'Hello World'.
5. We create an object that stores information about the font type and size we want to use.
6. We use this object to *render* text on the surface, using the render method, whose signature is *render(text, anti-*